IIMT3642: Managing and Mining Big Data - Large Graph & Link Analysis

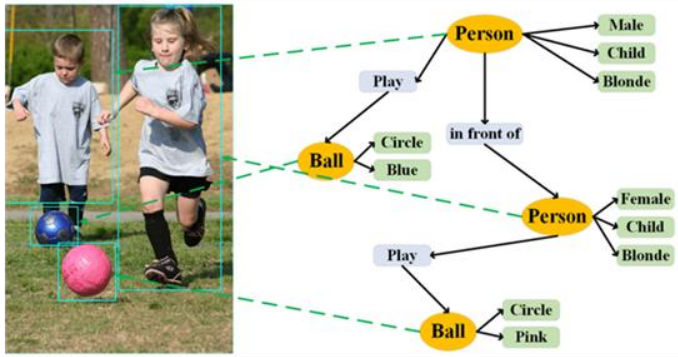
Zhepeng (Lionel) Li, PhD
Associate Professor
Innovation and Information Management

Graph

A representational structure that describes entities with relations

Social Network
(virtual)

Traffic Graph
(physical)



Scene Graph
(semantic)

Credits to Prof. Jure Leskovec, Anand Rajaraman, Jeff Ullman for many slides used in this lecture. More details are available at http://www.mmds.org/.

Large-scale Graph Data:

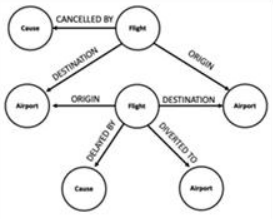
- World-wide Social Network

Graph Data: Social Networks

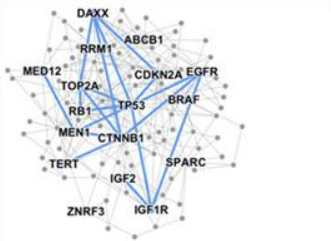
Facebook social graph

4-degrees of separation [Backstrom-Boldi-Rosa-Ugander-Vigna, 2011]

Graph is a ubiquitous organization of entities in reality



Event Graphs



Disease Pathways

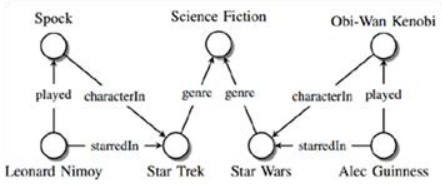


Image credit: Maximilian Nickel et al

Knowledge Graphs

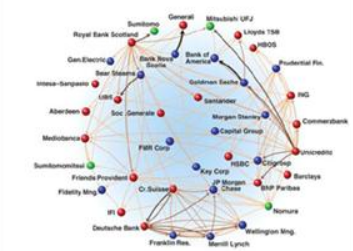


Image credit: Science

Economic Networks

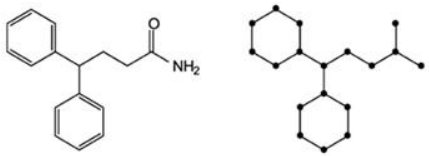
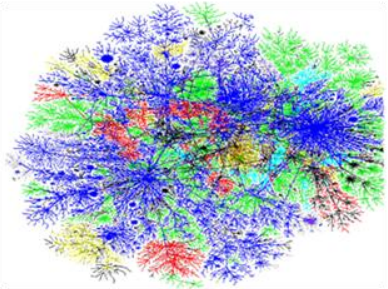
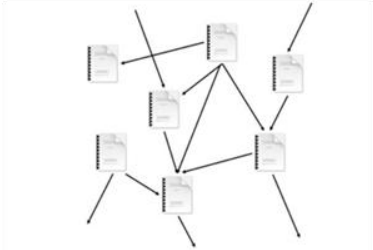


Image credit: MDPI

Molecules



Internet Topology



Citation Networks

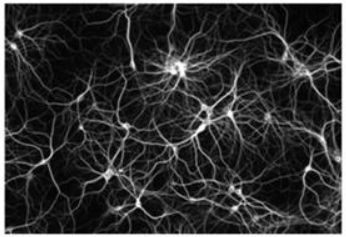
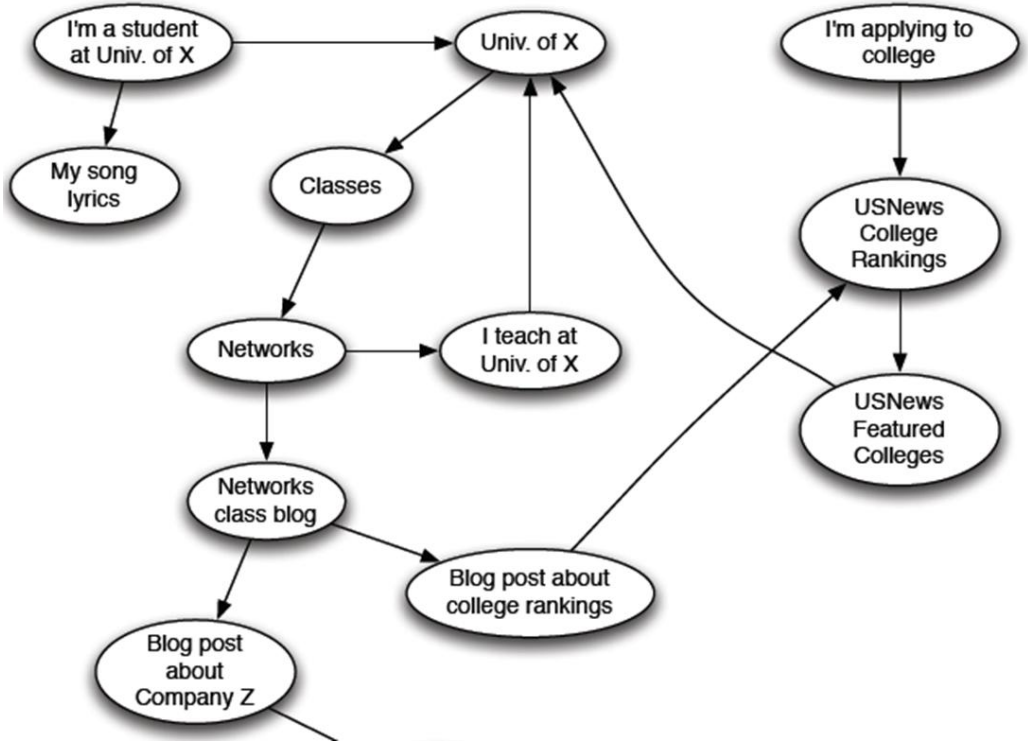


Image credit: The Conversation

Networks of Neurons

Web as a directed graph:

- Nodes: Webpages
- Edges: Hyperlinks



A General Question

How to organize the web?

- First try: Human curated Web directories
  - Yahoo!
- Second try: Web search
  - Information Retrieval: Find relevant docs in a small and trusted set
  - <u>Issue:</u> Web space is huge, full of untrusted documents, low-quality information, web spam etc.

Two challenges of Web search

HKU BUSINESS SCHOOL 港大經管學院

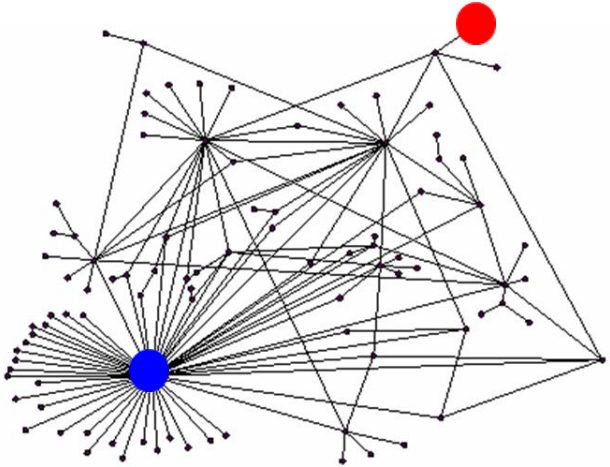
Problem for effective Web search (for google):

- (1) Web contains many sources of information, whom to trust?
  - Observation: Trustworthy pages may point to each other!
- (2) What is the best answer to a generic query such as “newspaper”?
  - No single right answer
  - Observation: Pages that actually know about newspaper might all be pointing to many newspapers

Link Structural Information Are Useful

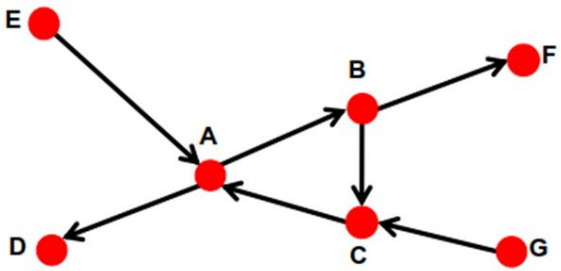
Ranking Nodes on the Graph:

- All web pages are not equally “important”
  - www.hku.hk vs. https://SomeNobody.github.io/
- There is large diversity in the web-graph node connectivity.
  - Let’s rank the pages by the link structure!

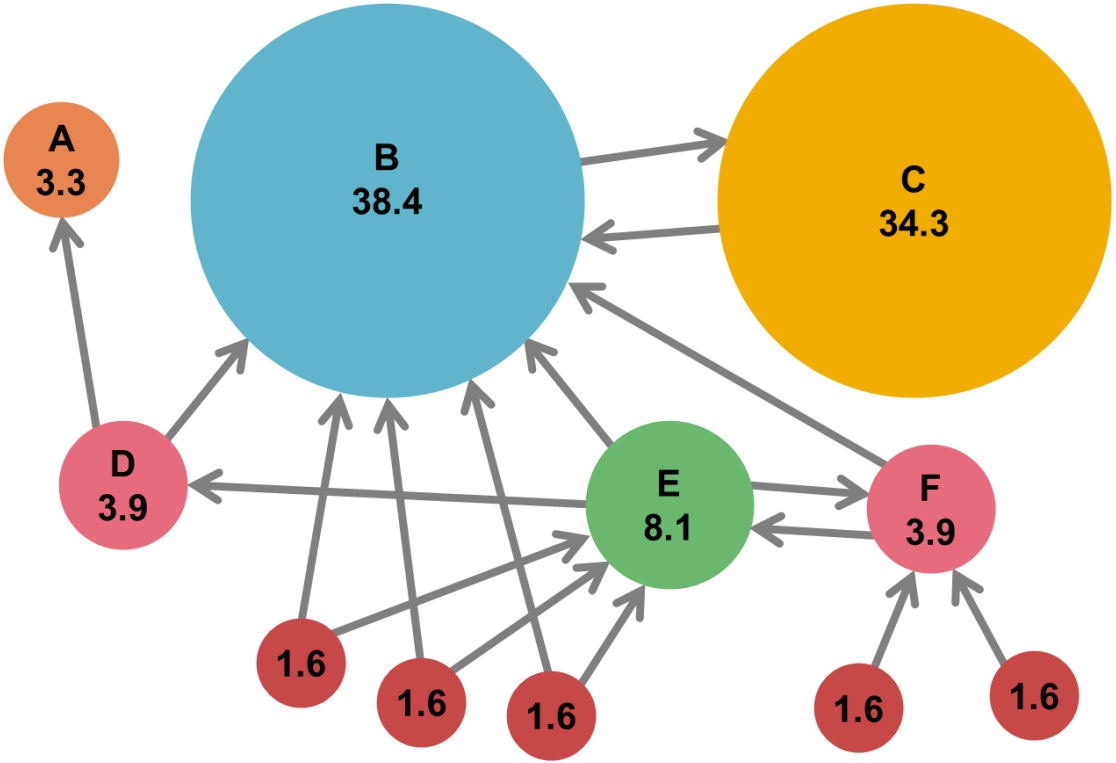


Link Analysis

- Link analysis approaches are adopted to address Web search problem
  - PageRank
  - Topic-specific (Personalized) PageRank
- The idea is to treat linkage as votes
  - Incoming links (in-links; hyperlinks that point to the focal node from other nodes) are considered recognition of importance from others
  - Outgoing links (out-links; hyperlinks pointing from the focal node to other nodes) are to recognize others
- In-links are NOT equal
  - Links from influential pages should be more important
  - When such incoming links are further directed onto other nodes, **recursion arises.**

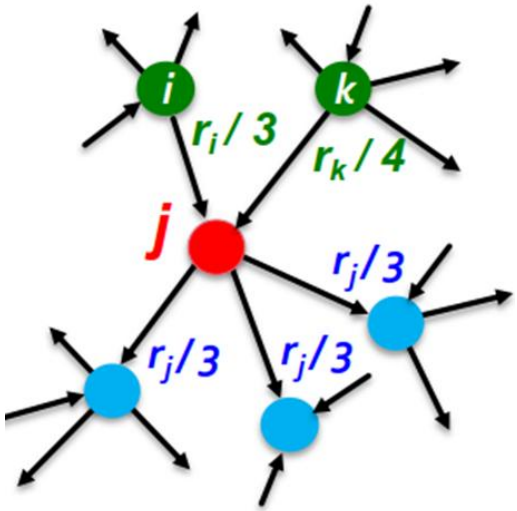


Example: PageRank Scores



The Flow Equations

- PageRank – a vote from important page is worth more
  - Each link's vote is proportional to the importance of its source page
  - If page i with important r_i has d_i out-links, each link gets $\frac{r_i}{d_i}$ votes
  - Page $j's$ own importance r_j is the sum of the votes over its in-links



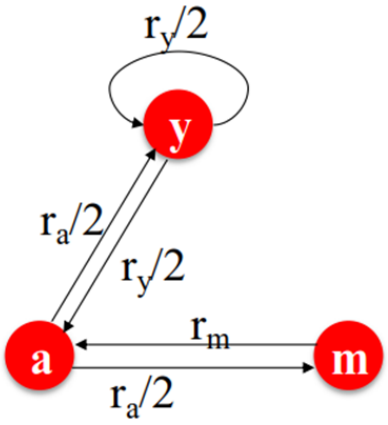
$$r_j = r_i/3 + r_k/4$$

PageRank

- Define "rank" r_j for node j

$$r_j = \sum_{i \to j} \frac{r_i}{d_i}$$

Where d_i is the number of out-links at node i



"Flow" equations:

$r_y = r_y/2 + r_a/2$

$r_a = r_y/2 + r_m$

$r_m = r_a/2$

Solving the Flow Equations:

- There are 3 equations, 3 unknowns, no constants
  - No unique solution
  - All solutions equivalent modulo the scale factor

Flow equations:

$$r_y = r_y/2 + r_a/2$$
$$r_a = r_y/2 + r_m$$
$$r_m = r_a/2$$

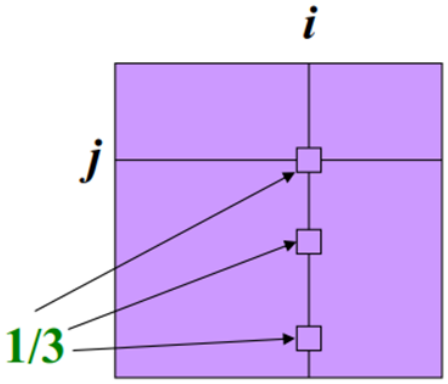
- One additional constraint is desired to force the uniqueness in solution
  - $r_y + r_a + r_m = 1$
  - **Solution:** $r_y = \frac{2}{5}$, $r_a = \frac{2}{5}$, $r_m = \frac{1}{5}$

- Traditional method works for small examples, but not efficient for large web-size graphs. We need a new formulation!

The Matrix Formulation

- PageRank – Matrix formulation
  - Stochastic adjacency matrix M
    - d_i - outdegree of node i
    - Given directed link $i \rightarrow j$, corresponding entry $M_{ji} = \frac{1}{d_i}$
    - Each column in M sums to 1
  - Rank vector $\boldsymbol{r}$ – one entry per node
    - r_i is the importance score of page node i
    - $\sum_i r_i = 1$
  - The flow equation is

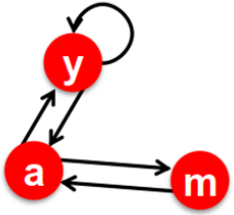
$$\boldsymbol{r} = \boldsymbol{M} \cdot \boldsymbol{r} \Leftrightarrow r_j = \sum_{i \rightarrow j} \frac{1}{d_i} \times r_i$$



Stochastic Matrix *M*

❑ Example: an instance of 3-node graph

- Stochastic matrix



	r_y	r_a	r_m
r_y	½	½	0
r_a	½	0	1
r_m	0	½	0

- Flow equations for recursive updating

$$r_y = r_y/2 + r_a/2$$
$$r_a = r_y/2 + r_m$$
$$r_m = r_a/2$$

$$\underbrace{\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix}}_{r} = \underbrace{\begin{bmatrix} ½ & ½ & 0 \\ ½ & 0 & 1 \\ 0 & ½ & 0 \end{bmatrix}}_{M} \underbrace{\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix}}_{r}$$

Converge to r^*…

- The flow equations can be written as

$$\boldsymbol{r} = \boldsymbol{M} \cdot \boldsymbol{r}$$

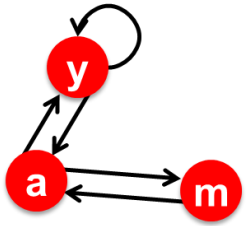
NOTE: ***x*** is an eigenvector with the corresponding eigenvalue **λ** if:

$$\boldsymbol{Ax} = \boldsymbol{\lambda x}$$

  - By definition, the **rank vector** ***r*** is an **eigenvector** of the stochastic web matrix ***M***
- In fact, ***r*** is the first or principal eigenvector that corresponds to eigenvalue ***1***
  - Quick fact: Largest eigenvalue of **M** is **1** since **M** is column stochastic (with non-negative entries).
  - Proof details are available at this link: https://mikespivey.wordpress.com/2013/01/17/eigenvalue-stochasti/

- We can now efficiently solve for ***r***
  - The method is called Power Iteration

Example:



	y	a	m
y	½	½	0
a	½	0	1
m	0	½	0

$$r = M \cdot r$$

$$r_y = r_y/2 + r_a/2$$
$$r_a = r_y/2 + r_m$$
$$r_m = r_a/2$$

$$\begin{bmatrix} y \\ a \\ m \end{bmatrix} = \begin{bmatrix} ½ & ½ & 0 \\ ½ & 0 & 1 \\ 0 & ½ & 0 \end{bmatrix} \begin{bmatrix} y \\ a \\ m \end{bmatrix}$$

Power Iteration Method

- Given a web graph with *n* nodes, where the nodes are pages and edges are hyperlinks
- Power iteration: a simple iterative scheme
  - Suppose there are *N* web pages
  - Initialize: $\mathbf{r}^{(0)} = [1/N,\ldots,1/N]^T$
  - Iterate: $\mathbf{r}^{(t+1)} = \mathbf{M} \cdot \mathbf{r}^{(t)}$
  - Stop when $|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}|_1 < \varepsilon$

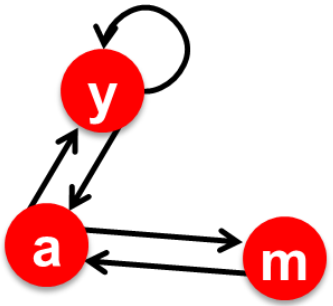
$$r_j^{(t+1)} = \sum_{i \to j} \frac{r_i^{(t)}}{d_i}$$

d_i …. out-degree of node i

$|\mathbf{x}|_1 = \sum_{1 \le i \le N} |x_i|$ is the $\mathbf{L_1}$ norm
Can use any other vector norm, e.g., Euclidean

PageRank: Solution

- Power Iteration:
  - Set $r_j = 1/N$
  - 1: $r'_j = \sum_{i \to j} \frac{r_i}{d_i}$
  - 2: $r = r'$
  - Goto Step **1**



	y	a	m
y	½	½	0
a	½	0	1
m	0	½	0

$$r_y = r_y/2 + r_a/2$$
$$r_a = r_y/2 + r_m$$
$$r_m = r_a/2$$

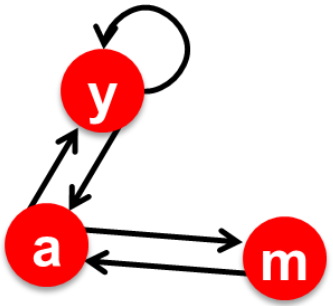
- Example:

$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{matrix} 1/3 \\ 1/3 \\ 1/3 \end{matrix}$$

Iteration 0, 1, 2, …

PageRank: Solution

- Power Iteration:
  - Set $r_j = 1/N$
  - 1: $r'_j = \sum_{i \to j} \frac{r_i}{d_i}$
  - 2: $r = r'$
  - Goto Step **1**



	y	a	m
y	½	½	0
a	½	0	1
m	0	½	0

$$r_y = r_y/2 + r_a/2$$
$$r_a = r_y/2 + r_m$$
$$r_m = r_a/2$$

- Example:

$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{matrix} 1/3 & 1/3 & 5/12 & 9/24 & & 6/15 \\ 1/3 & 3/6 & 1/3 & 11/24 & \dots & 6/15 \\ 1/3 & 1/6 & 3/12 & 1/6 & & 3/15 \end{matrix}$$

Iteration 0, 1, 2, …

Why does Power Iteration Work?

- Power Iteration:

  A method that finds the dominant eigenvector (the vector corresponding to the largest eigenvalue)

  - $\boldsymbol{r}^{(1)} = \boldsymbol{M} \cdot \boldsymbol{r}^{(0)}$
  - $\boldsymbol{r}^{(2)} = \boldsymbol{M} \cdot \boldsymbol{r}^{(1)} = \boldsymbol{M}\left(\boldsymbol{M}\boldsymbol{r}^{(1)}\right) = \boldsymbol{M}^2 \cdot \boldsymbol{r}^{(0)}$
  - $\boldsymbol{r}^{(3)} = \boldsymbol{M} \cdot \boldsymbol{r}^{(2)} = \boldsymbol{M}\left(\boldsymbol{M}^2\boldsymbol{r}^{(0)}\right) = \boldsymbol{M}^3 \cdot \boldsymbol{r}^{(0)}$

- Claim:
  Sequence $\boldsymbol{M} \cdot \boldsymbol{r}^{(0)}, \boldsymbol{M}^2 \cdot \boldsymbol{r}^{(0)}, \ldots \boldsymbol{M}^k \cdot \boldsymbol{r}^{(0)}, \ldots$ approaches a multiple of the dominant eigenvector of $\boldsymbol{M}$

Proof of the claim:

- Assume ***M*** has *n* linearly independent eigenvectors $x_1, x_2, \ldots, x_n$ with corresponding eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$, where $\lambda_1 > \lambda_2 > \ldots > \lambda_n$

- Vectors $x_1, x_2, \ldots, x_n$ form a basis and thus we can write:

$$r^{(0)} = c_1 x_1 + c_2 x_2 + \cdots + c_n x_n$$

$$\begin{aligned} \boldsymbol{M r^{(0)}} &= \boldsymbol{M(c_1\, x_1 + c_2\, x_2 + \cdots + c_n\, x_n)} \\ &= c_1(Mx_1) + c_2(Mx_2) + \cdots + c_n(Mx_n) \\ &= c_1(\lambda_1 x_1) + c_2(\lambda_2 x_2) + \cdots + c_n(\lambda_n x_n) \end{aligned}$$

Repeated multiplication on both sides produces

$$M^k r^{(0)} = c_1(\lambda_1^k x_1) + c_2(\lambda_2^k x_2) + \cdots + c_n(\lambda_n^k x_n)$$

(continued) Proof of the claim:

- Repeated multiplication on both sides produces

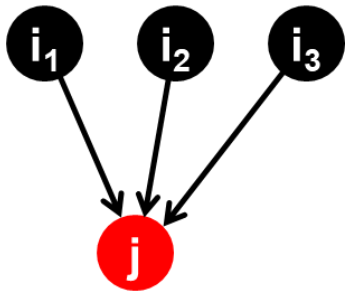
$$
\begin{aligned}
M^k r^{(0)} &= c_1(\lambda_1^k x_1) + c_2(\lambda_2^k x_2) + \cdots + c_n(\lambda_n^k x_n) \\
&= \lambda_1^k \left[c_1 x_1 + c_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k x_2 + \cdots + c_n \left(\frac{\lambda_n}{\lambda_1}\right)^k x_n \right]
\end{aligned}
$$

- Since $\lambda_1 > \lambda_2 > \cdots > \lambda_n$ then $\left(\frac{\lambda_i}{\lambda_1}\right)^k = 0 \text{ as } k \to \infty \text{ for all } i = 2, \ldots, n$
- Thus, we have $\boldsymbol{M^k r^{(0)} \approx c_1(\lambda_1^k x_1)}$

Note if c_1=0 then the method won't converge

Alternative Interpretation: Random Walk

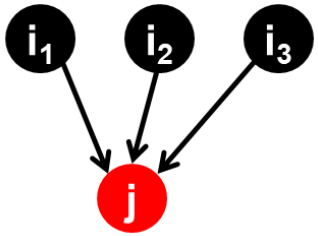
- Imagine a random web surfer:
  - At any time t, surfer is on some page i
  - At time $t+1$, the surfer follows an out-link from i uniformly at random
  - Ends up on some page j linked from i
  - Process repeats indefinitely
- Let:
  - $p(t)$ denote a vector in which the ith element is the prob. that the surfer is at page i at time t
  - So, $p(t)$ is a probability distribution over pages



$$r_j = \sum_{i \to j} \frac{r_i}{d_{out}(i)}$$

- Where is the surfer at time t+1?
  - Follows a link uniformly at random

$$\boldsymbol{p}(\boldsymbol{t}+\mathbf{1}) = \boldsymbol{M}\cdot\boldsymbol{p}(\boldsymbol{t})$$



$$p(t+1) = \mathrm{M}\cdot p(t)$$

  - Suppose the random walk reaches a state

$\boldsymbol{p}(\boldsymbol{t}+\mathbf{1}) = \boldsymbol{M}\cdot\boldsymbol{p}(\boldsymbol{t}) = \boldsymbol{p}(\boldsymbol{t})$, where $\boldsymbol{p}(\boldsymbol{t})$ is stationary distribution of a random walk

- Our original rank vector $\boldsymbol{r}$ satisfies $\boldsymbol{r} = \boldsymbol{M}\cdot\boldsymbol{r}$
  - So, $\boldsymbol{r}$ can be taken as a stationary distribution for the random walk

Solution Existence and Uniqueness

- **A central result from the theory of random walks (a.k.a. Markov processes):**

For graphs that satisfy **certain conditions**, the **stationary distribution is unique** and eventually will be reached no matter what the initial probability distribution at time **t = 0**

Note: Given the transition matrix is irreducible and aperiodic, the markov process will converge to a unique stationary state

PageRank: The Google Formulation

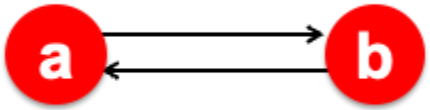
$$r_j^{(t+1)} = \sum_{i \to j} \frac{r_i^{(t)}}{\mathrm{d_i}}$$

or equivalently

$$r = Mr$$

- Does this converge?
- Does it converge to what we want?
- Are results reasonable?

- Does this converge?



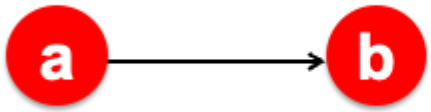
$$r_j^{(t+1)} = \sum_{i \to j} \frac{r_i^{(t)}}{d_i}$$

Example:

$$\begin{matrix} r_a \\ r_b \end{matrix} = \begin{matrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{matrix}$$

Iteration 0, 1, 2, …

- Does this converge to what we want?



$$r_j^{(t+1)} = \sum_{i \to j} \frac{r_i^{(t)}}{d_i}$$

Example:

$$\begin{matrix} r_a \\ r_b \end{matrix} = \begin{matrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{matrix}$$

Iteration 0, 1, 2, …

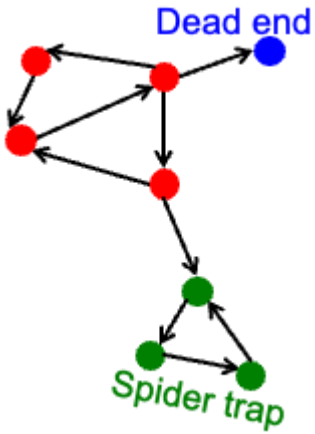
Two realistic problems:

(1) Some pages are dead ends (have no out-links)

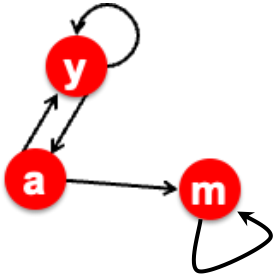
- Random walk has “nowhere” to go to
- Such pages cause importance to “leak out”

(2) Spider traps: (all out-links are within the group)

- Random walker gets “stuck” in a trap
- And eventually spider traps absorb all importance



Problem: Spider Traps



	y	a	m
y	½	½	0
a	½	0	0
m	0	½	1

$$r_y = r_y/2 + r_a/2$$
$$r_a = r_y/2$$
$$r_m = r_a/2 + r_m$$

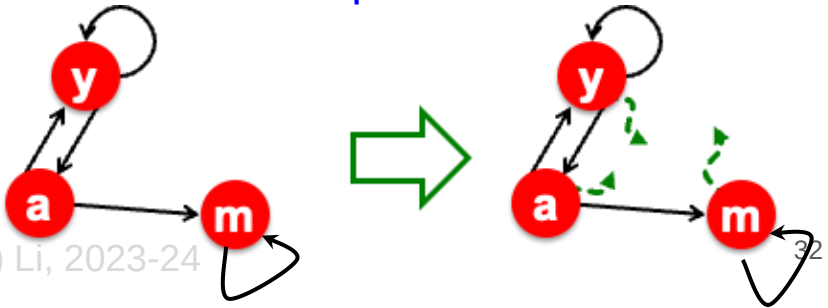
Example:

$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{matrix} 1/3 & 2/6 & 3/12 & 5/24 & & 0 \\ 1/3 & 1/6 & 2/12 & 3/24 & \dots & 0 \\ 1/3 & 3/6 & 7/12 & 16/24 & & 1 \end{matrix}$$

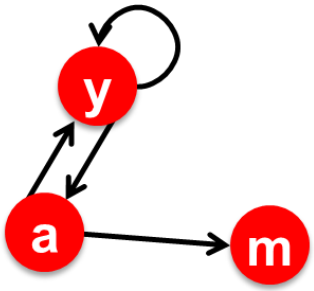
Iteration 0, 1, 2, …

Solution: Teleports

- The Google solution for spider traps: At each time step, the random surfer has two options
  - With prob. β, follow a link at random
  - With prob. 1-β, jump to some random page regardless of whether a link exists
  - Common values for β are in the range 0.8 to 0.9
- Surfer will teleport out of spider trap within a few time steps



Problem: Dead Ends



	y	a	m
y	½	½	0
a	½	0	0
m	0	½	0

Example

$$r_y = r_y/2 + r_a/2$$
$$r_a = r_y/2$$
$$r_m = r_a/2$$

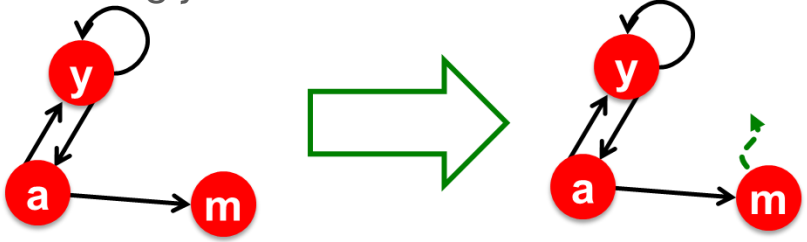
$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{matrix} 1/3 & 2/6 & 3/12 & 5/24 & & 0 \\ 1/3 & 1/6 & 2/12 & 3/24 & \dots & 0 \\ 1/3 & 1/6 & 1/12 & 2/24 & & 0 \end{matrix}$$

Iteration 0, 1, 2, …

Note: the PageRank “leaks” out since the matrix is no longer stochastic

Solution: Teleports again!

- Follow random teleport links with probability 1.0 once the surfer arrives at dead-ends
  - Adjust matrix accordingly



	y	a	m
y	½	½	0
a	½	0	0
m	0	½	0

	y	a	m
y	½	½	⅓
a	½	0	⅓
m	0	½	⅓

Why are dead-ends and spider traps a problem and why do teleports solve the problem?

- Spider-traps are not problematic in theory
  - But with spider traps, PageRank scores are not what we want
  - Solution: Never get stuck in a spider trap by teleporting out of it in a finite number of steps
- Dead-ends are problematic in theory
  - The matrix is not column stochastic so our initial assumptions are not met
  - Solution: Make matrix column stochastic by always teleporting when there is nowhere else to go

Random Teleports: The solution at Google

- At each step, random surfer has two options:
  - With probability β, follow a link at random
  - With probability 1-β, jump to some random page
- PageRank equation (Brin and Page 1998)

$$r_j = \sum_{i \to j} \beta \frac{r_i}{d_i} + (1-\beta)\frac{1}{N}$$

d_i ... out-degree of node i

N is the number of all nodes

This formulation assumes that $\boldsymbol{M}$ has no dead ends. We can either preprocess matrix $\boldsymbol{M}$ to remove all dead ends or explicitly follow random teleport links with probability 1.0 from dead-ends.

- PageRank equation (Brin and Page 1998)

$$r_j = \sum_{i \to j} \beta \frac{r_i}{d_i} + (1-\beta)\frac{1}{N}$$

d_i ... out-degree of node i

- The Google Matrix A:

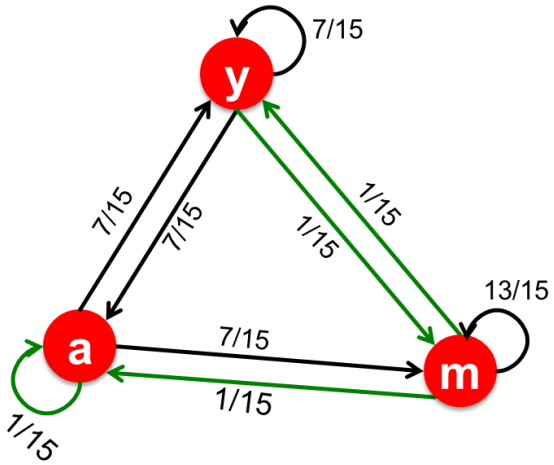
$$A = \beta M + (1-\beta)\left[\frac{1}{N}\right]_{N \times N}$$

$[1/N]_{NxN}$...N by N matrix where all entries are 1/N

- We have a recursive problem: $\boldsymbol{r} = \boldsymbol{A} \cdot \boldsymbol{r} = \beta \boldsymbol{M}\boldsymbol{r} + \left[\frac{1-\beta}{N}\right]_N$ Second term is a vector with N entries
  - The power method still applies!

In practice, we set β in the range [0.8,0.9]

Example: Random Teleport (β=0.8)



$$0.8 \begin{array}{c} \mathbf{M} \\ \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 0 \\ 0 & 1/2 & 1 \end{bmatrix} \end{array} + 0.2 \begin{array}{c} [1/N]_{NxN} \\ \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix} \end{array}$$

$$\begin{array}{c} \\ y \\ a \\ m \end{array} \begin{array}{c} \\ \begin{bmatrix} 7/15 & 7/15 & 1/15 \\ 7/15 & 1/15 & 1/15 \\ 1/15 & 7/15 & 13/15 \end{bmatrix} \end{array}$$

A

$$\begin{array}{c} y \\ a \\ m \end{array} = \begin{array}{ccccc} 1/3 & 0.33 & 0.24 & 0.26 & \\ 1/3 & 0.20 & 0.20 & 0.18 & \ldots \\ 1/3 & 0.46 & 0.52 & 0.56 & \end{array} \begin{array}{c} 7/33 \\ 5/33 \\ 21/33 \end{array}$$

Discussion: Computing PageRank

- The essential task is matrix-vector multiplication

$$r^{new} = A \cdot r^{old}$$

- Easy if we have enough main memory to hold **A**, r^{new} and r^{old}
- For instance, let us consider N = 1 billion web pages
  - We need 4 bytes for each entry
  - 2 billion entries of the two vectors, approximately 8GB
  - Matrix **A** has N^2 entries
    - 10^{18} is a huge number!

$$\mathbf{A} = \beta \cdot \mathbf{M} + (1-\beta)\,[1/N]_{N \times N}$$

$$A = 0.8 \begin{bmatrix} ½ & ½ & 0 \\ ½ & 0 & 0 \\ 0 & ½ & 1 \end{bmatrix} + 0.2 \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix}$$

$$= \begin{bmatrix} 7/15 & 7/15 & 1/15 \\ 7/15 & 1/15 & 1/15 \\ 1/15 & 7/15 & 13/15 \end{bmatrix}$$

Sparse Matrix Encoding

Encodes sparse matrix using only non-zero entries

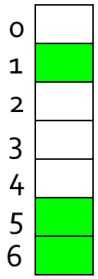
- Space proportional roughly to number of links
- Say 10N, or 4*10*1 billion = 40GB
- Still won't fit in memory, but will fit on disk

source node	degree	destination nodes
0	3	1, 5, 7
1	5	17, 64, 113, 117, 245
2	2	13, 23

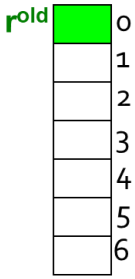
Basic Algorithm: Power Iteration

- Assume enough RAM to fit r^{new} into memory
  - Store r^{old} and matrix M on hard disk
- The update step of power-iteration is

Initialize all entries of $r^{new} = (1-\beta) / N$
For each page i (of out-degree d_i):
  Read into memory: i, d_i, $dest_1$, …, $dest_{d_i}$, $r^{old}(i)$
  For $j = 1 \ldots d_i$
    $r^{new}(dest_j) \mathrel{+}= \beta\, r^{old}(i) / d_i$



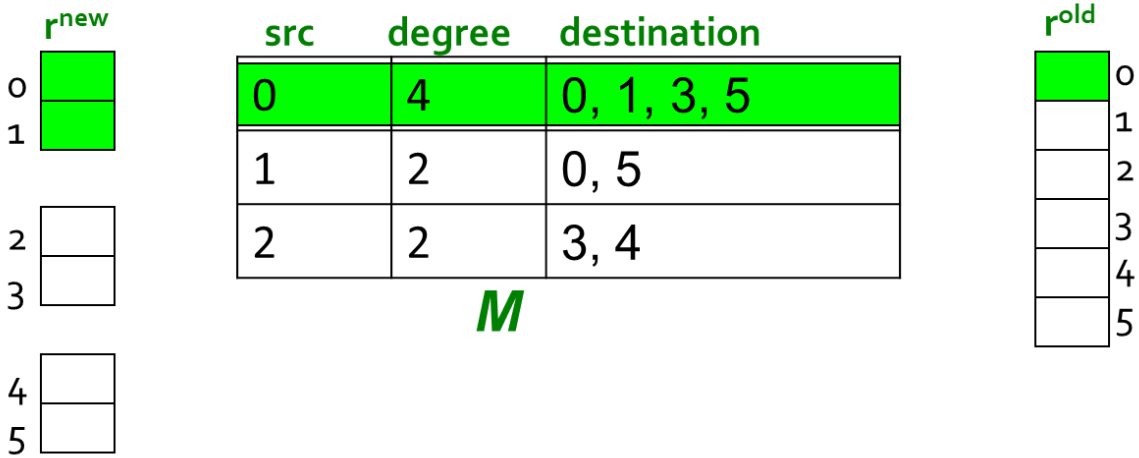
source	degree	destination
0	3	1, 5, 6
1	4	17, 64, 113, 117
2	2	13, 23



Complexity Analysis: Update Once for All

- Assume enough RAM to fit $\boldsymbol{r}^{new}$ into memory
  - Store r^{old} and matrix $\boldsymbol{M}$ on disk
- In each iteration, we have to:
  - Read $\boldsymbol{r}^{old}$ and $\boldsymbol{M}$
  - Write $\boldsymbol{r}^{new}$ back to disk
  - Cost per iteration of Power method: $2 \times |\boldsymbol{r}| + |\boldsymbol{M}|$
- Question:
  - What if we could not even fit $\boldsymbol{r}^{new}$ in memory?

Advanced Algorithm: Block-based Update

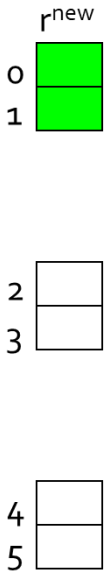


- Break $\boldsymbol{r}^{new}$ into ***k*** blocks that fit in memory
- Scan ***M*** and $\boldsymbol{r}^{old}$ once for each block

Complexity Analysis: Block Update

- Block-based Update
  - Break $\boldsymbol{r}^{new}$ into $\boldsymbol{k}$ blocks that fit in memory
  - Scan **M** and $\boldsymbol{r}^{old}$ once for each block
- Computational Cost
  - $\boldsymbol{k}$ scans of $\boldsymbol{M}$ and $\boldsymbol{r}^{old}$
  - Cost per iteration of Power method: $\boldsymbol{k} \times (|\boldsymbol{M}| + |\boldsymbol{r}|) + |\boldsymbol{r}| = \boldsymbol{k}|\boldsymbol{M}| + (\boldsymbol{k}+1)|\boldsymbol{r}|$
- Can we do any better?
  - Hint: **M** is much bigger than **r** (approx 10-20x), so we must avoid reading it $\boldsymbol{k}$ times per iteration

Advanced Algorithm: Block-stripe Update



src	degree	destination
0	4	0, 1
1	3	0
2	2	1

0	4	3
2	2	3

0	4	5
1	3	5
2	2	4

Break *M* into stripes! Each stripe contains only destination nodes in the corresponding block of r^{new}

Complexity Analysis: Block-stripe Update

- Break ***M*** into stripes
  - Each stripe contains only destination nodes in the corresponding block of $\boldsymbol{r}^{new}$
- Some additional overhead ε per stripe
  - But it is usually worth it
- Cost per iteration of Power method: $|\boldsymbol{M}| \times (1+\varepsilon) + (\boldsymbol{k}+1) \times |\boldsymbol{r}|$